

DATABASE CREATION AND TABLE IMPLEMENTATION

ASSIGNMENT 5

DATABASE MANAGEMENT SYSTEM

ALLIANCE UNIVERSITY

Department of Computer Applications

SUMMITED BY: TINAKAR C

REGISTER NO: 2511021240003

COURSE: BCA

YEAR: 2ND YEAR

INTRODUCTION

Database creation and table implementation are important steps in developing a database management system. SQL Data Definition Language commands create databases, define tables, specify data types, and establish relationships. This assignment implements the Online Shopping Management System in MySQL using eight tables.

1. CREATING THE DATABASE

```
CREATE DATABASE Online_Shopping_DB;
USE Online_Shopping_DB;
```

CREATE DATABASE creates the database, and USE selects it for creating and managing tables.

2. CREATING THE REQUIRED TABLES

TABLE 1: CUSTOMER

```
CREATE TABLE CUSTOMER (
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL,
  phone VARCHAR(15),
  address VARCHAR(200)
);
```

Explanation: customer_id uniquely identifies each customer. The table stores name, email, phone, and address.

TABLE 2: CATEGORY

```
CREATE TABLE CATEGORY (
  category_id INT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL,
  description VARCHAR(200)
);
```

Explanation: category_id uniquely identifies a product category.

TABLE 3: SUPPLIER

```
CREATE TABLE SUPPLIER (
  supplier_id INT PRIMARY KEY,
  supplier_name VARCHAR(100) NOT NULL,
  email VARCHAR(100),
  phone VARCHAR(15)
);
```

Explanation: supplier_id uniquely identifies a supplier and stores contact information.

TABLE 4: PRODUCT

```
CREATE TABLE PRODUCT (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
```

```
  price DECIMAL(10,2) NOT NULL,
  stock_quantity INT NOT NULL,
  category_id INT,
  supplier_id INT,
  FOREIGN KEY (category_id) REFERENCES CATEGORY(category_id),
  FOREIGN KEY (supplier_id) REFERENCES SUPPLIER(supplier_id)
);
```

Explanation: product_id identifies each product. Category_ID and Supplier_ID connect products to their category and supplier.

TABLE 5: ORDERS

```
CREATE TABLE ORDERS (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  order_date DATE NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  order_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id)
);
```

Explanation: order_id identifies each order. customer_id connects the order to its customer.

TABLE 6: ORDER_ITEM

```
CREATE TABLE ORDER_ITEM (
  order_item_id INT PRIMARY KEY,
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES ORDERS(order_id),
  FOREIGN KEY (product_id) REFERENCES PRODUCT(product_id)
);
```

Explanation: Each record stores a product included in an order, including quantity and unit price.

TABLE 7: PAYMENT

```
CREATE TABLE PAYMENT (
  payment_id INT PRIMARY KEY,
  order_id INT NOT NULL,
  payment_date DATE,
  payment_method VARCHAR(30),
  amount DECIMAL(10,2) NOT NULL,
  payment_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES ORDERS(order_id)
);
```

Explanation: payment_id identifies a payment, and order_id links it to the corresponding order.

TABLE 8: DELIVERY

```
CREATE TABLE DELIVERY (
  delivery_id INT PRIMARY KEY,
  order_id INT NOT NULL,
  delivery_date DATE,
  delivery_address VARCHAR(200) NOT NULL,
```

```
  delivery_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES ORDERS(order_id)
);
```

Explanation: delivery_id identifies the delivery record. The table stores expected or completed date, address, and status.

3. DEFINING RELATIONSHIPS BETWEEN TABLES

Relationship	Type	Purpose
CUSTOMER → ORDERS	One-to-Many	One customer can place multiple orders.
CATEGORY → PRODUCT	One-to-Many	One category can contain multiple products.
SUPPLIER → PRODUCT	One-to-Many	One supplier can supply multiple products.
ORDERS → ORDER_ITEM	One-to-Many	One order can contain multiple products.
PRODUCT → ORDER_ITEM	One-to-Many	A product can appear in many order-item records.
ORDERS → PAYMENT	One-to-One intended	Each order is intended to have one payment record.
ORDERS → DELIVERY	One-to-One intended	Each order is intended to have one delivery record.

4. VERIFYING TABLE CREATION

```
SHOW TABLES;
DESCRIBE CUSTOMER;
DESCRIBE PRODUCT;
DESCRIBE ORDERS;
```

Expected output for SHOW TABLES:

Tables_in_Online_Shopping_DB
CATEGORY
CUSTOMER
DELIVERY
ORDER_ITEM
ORDERS
PAYMENT
PRODUCT
SUPPLIER

DESCRIBE displays column names, data types, key information, and nullability for the selected table. The same command can be used with the other tables.

CONCLUSION

Eight tables were created to manage customers, categories, products, suppliers, orders, order items, payments, and deliveries. Primary keys identify records and foreign keys establish relationships. This design provides a foundation for inserting sample data and running SQL queries.